

An evaluation of the degree of agility in six agile methods and its applicability for method engineering

A. Qumer, B. Henderson-Sellers *

Faculty of Information Technology, University of Technology, Sydney, PO Box 123, Broadway, NSW 2007, Australia

Received 13 November 2006; received in revised form 18 January 2007; accepted 4 February 2007

Available online 11 February 2007

Abstract

While agile methods are in use in industry, little research has been undertaken into what is meant by agility and how a supposed agile method can be evaluated with regard to its veracity to belong to this category of software development methodological approaches. Here, an analytical framework, called 4-DAT, is developed and applied to six well-known agile methods and, for comparison, two traditional methods. The results indicate the degree of agility to be found in each method, from which a judgement can be made as to whether the appellation of “agile” to that method is appropriate. This information is shown to be useful, for example, when constructing a methodology from method fragments (method engineering) and when comparing agile and traditional methods.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Agile methods; Method engineering; Software engineering; Methodology

1. Introduction

As a reaction to so-called “heavyweight” or plan-based methodologies [1], many practitioners have adopted the recently introduced ideas of agility [2]. Use of an agile method, under optimum conditions, can indeed create a high quality environment and high quality product. Its focus on the people rather than on reporting deliverables is often seen as a welcome shift of balance towards the most important factor in software development: the personnel involved. However, little objective evidence exists as to which of these many so-called agile methods really possesses the kind of agility envisioned in the Agile Manifesto [2]. Here, we introduce a new tool that can be used to assist in such an evaluation. For example, a manager may wish to choose between the various agile methods on offer; a method engineer may wish to include agile method fragments within their situational method.

This tool, called 4-DAT [3], provides a framework in which an assessment of four degrees of agility can be made for an existing methodology or for an about-to-constructed methodology. In Section 2, we define agility, based on earlier studies [4], then summarize the 4-DAT (based on [3]). The bulk of the paper then introduces novel results derived from the 4-DAT analysis of six methods, extending the brief analysis of Scrum and XP given in [5]. In Section 3, we evaluate two non-agile (traditional) methodologies in order to provide a contrast. We then conclude with a short discussion about how these ideas could be used in the construction of a brand new method using a method engineering approach, as an example of the applicability of the agility assessment results from this 4-DAT analysis.

2. Agile development methods

2.1. Defining agility

Although Cockburn [6] defines the core of agile methods as “the use of light-but-sufficient rules of project behaviour and the use of human-and communication-oriented rules” and the Agile Manifesto [2] provides agile principles and

* Corresponding author. Tel.: +61 2 9514 1687; fax: +61 2 9514 4535.
E-mail addresses: asif@it.uts.edu.au (A. Qumer), brian@it.uts.edu.au (B. Henderson-Sellers).

agile values that qualitatively characterize agile methods, there remains no widely-agreed, precise and comprehensive definition of agility. Based on a survey and assessment of the various contemporary definitions, Qumer and Henderson-Sellers [3,5] offer the following definition for the agility of any entity:

“Agility is a persistent behaviour or ability of a sensitive entity that exhibits flexibility to accommodate expected or unexpected changes rapidly, follows the shortest time span, uses economical, simple and quality instruments in a dynamic environment and applies updated prior knowledge and experience to learn from the internal and external environment.”

In order to benefit from this definition, these authors developed a four dimensional framework (4-DAT) to crystallize the key attributes of agility: flexibility, speed, leanness, learning and responsiveness. Flexibility is the ability to respond to change and leanness accentuates lower cost, reduced timeframe and quality production [7].

Consequently, by applying the above definition of agility to the notion of a software development methodology, we derive the definition of an “agile method” as:

“A software development method is said to be an agile software development method when a method is people focused, communications-oriented, flexible (ready to adapt to expected or unexpected change at any time), speedy (encourages rapid and iterative development of the product in small releases), lean (focuses on shortening timeframe and cost and on improved quality), responsive (reacts appropriately to expected and unexpected changes), and learning (focuses on improvement during and after product development)”.

2.2. Evaluating existing “agile methods” – the 4-DAT evaluation framework

Although there are a number of analytical tools and frameworks [8–12] that may be used to examine software development methods, the distinguishing feature of 4-DAT is that it evaluates methods from the perspectives of agility (called here the degree of agility), software process and lifecycle phases as well as how the methods fare in practice. 4-DAT specifically provides a mechanism to measure agility (degree of agility) of any method quantitatively at a specific level in a process and using specific practices.

The approach of 4-DAT is partly qualitative and partly quantitative. However, since the tool is generic, it may be used to analyze any other software development method as well (see Section 3). It will help to examine agile methods from four perspectives or “dimensions” (see Table 1 for details): method scope, agility characterization (based on the five key attributes introduced in Section 2.1), characterization of agile values (based on

those proposed in the Agile Manifesto [2]), and software process characterization [13]. A report generated with the help of 4-DAT will assist organizations in making decisions about the selection or adoption of an agile method or agile method fragments. Although there are currently four dimensions evaluated in the 4-DAT approach, it is in fact extensible – we can add or remove dimensions or items from the dimensions of 4-DAT, if found necessary in the future.

The method scope dimension is a set of key scope items, having been derived by distilling a set of key scope items from existing agile methods. This dimension checks the support of a method in terms of project size, team size, development style, code style, technology environment, physical environment, business culture and abstraction mechanism. This first dimension for the evaluation approach helps to compare the methods at a high level.

Agility characterization is identified as a second dimension of 4-DAT. It provides a set of agility features that are derived from the proposed agility definition (Section 2.1). Dimension 2 is used here to check the existence of agility in agile methods at both a process level and a method practices level. This is the only one of the four proposed dimensions of agility that is quantitative (details of the algorithms proposed are found in [3], reproduced here in Appendix A). A table is constructed for this dimension (Table 2) in which cell values of 0 or 1 are entered for each phase (for a high level assessment) or, at a more detailed level, for each individual practice (often called technique) [5]. The five agility features of flexibility (FY), speed (SD), leanness (LS), learning (LG) and responsiveness (RS) for each method fragment are considered and then the overall method total (and hence average degree of agility) can be calculated. Based on our quantitative evaluations of pre-existing (and well-known) agile methods such as XP and Scrum ([5] and see discussion below), we can offer as a ballpark figure a threshold value of around 0.5–0.6 for any constructed agile method to have sufficient measured agility to qualify for *consideration* as an agile method.

Agile values characterization, dimension 3, is a set of six agile values; four of them are provided by the Agile Manifesto [2] and the fifth agile value is provided by [14]. The sixth value of “keeping the process cost effective” was proposed in [3] based on the study of several agile methods. Dimension 3 examines the support of agile values in different practices of agile methods.

Dimension 4 is a set of four components characterizing the software process. There are two main components: product engineering process and process management process. A product engineering process has a further three categories: development process, project management process and support processes [13]. Dimension 4 examines the practices that support these four processes in agile methods.

Table 1
4-DAT's four dimensions (after [3])

Dimension 1 (Method Scope)	
<i>Scope</i>	<i>Description</i>
1. Project Size	Does the method specify support for small, medium or large projects (business or other)?
2. Team size	Does the method support for small or large teams (single or multiple teams)?
3. Development style	Which development style (iterative, rapid) does the method cover?
4. Code style	Does the method specify code style (simple or complex)?
5. Technology environment	Which technology environment (tools, compilers) does the method specify?
6. Physical environment	Which physical environment (co-located or distributed) does the method specify?
7. Business culture	What type of business culture (collaborative, cooperative or non-collaborative) does the method specify?
8. Abstraction mechanism	Does the method specify an abstraction mechanism (object-oriented, agent-oriented)?
Dimension 2 (Agility Characterization)	
<i>Features</i>	<i>Description</i>
1. Flexibility	Does the method accommodate expected or unexpected changes?
2. Speed	Does the method produce results quickly?
3. Leanness	Does the method follow the shortest time span, use economical, simple and quality instruments for production?
4. Learning	Does the method apply updated prior knowledge and experience to create a learning environment?
5. Responsiveness	Does the method exhibit sensitiveness?
Dimension 3 (Agile Value Characterization)	
<i>Agile values</i>	<i>Description</i>
1. Individuals and interactions over processes and tools	Which practices value people and interaction over processes and tools?
2. Working software over comprehensive documentation	Which practices value working software over comprehensive documentation?
3. Customer collaboration over contract negotiation	Which practices value customer collaboration over contract negotiation?
4. Responding to change over following a plan	Which practices value responding to change over following a plan?
5. Keeping the process agile	Which practices help in keeping the process agile?
6. Keeping the process cost effective	Which practices help in keeping the process cost effective?
Dimension 4 (Software Process Characterization)	
<i>Process</i>	<i>Description</i>
1. Development process	Which practices cover the main life cycle process and testing (Quality Assurance)?
2. Project management process	Which practices cover the overall management of the project?
3. Software configuration control process/Support process	Which practices cover the process that enable configuration management?
4. Process management process	Which practices cover the process that is required to manage the process itself?

Table 2

Schematic table for the calculation of the agility of x individual phases and y practices (a.k.a. techniques) and the overall agility of the constructed method

	Agility features					Total
	FY	SD	LS	LG	RS	
<i>Phases</i>						
Phase 1	0 or 1	0 or 1	0 or 1	0 or 1	0 or 1	0-5
Phase 2	0 or 1	0 or 1	0 or 1	0 or 1	0 or 1	0-5
Phase 3	0 or 1	0 or 1	0 or 1	0 or 1	0 or 1	0-5
etc.	0 or 1	0 or 1	0 or 1	0 or 1	0 or 1	0-5
Total	(0-x)	(0-x)	(0-x)	(0-x)	(0-x)	0-5*x
Degree of agility (high level)	(0-x)/x	(0-x)/x	(0-x)/x	(0-x)/x	(0-x)/x	Total divided by number of cells in table
<i>Practices</i>						
Practice 1	0 or 1	0 or 1	0 or 1	0 or 1	0 or 1	0-5
Practice 2	0 or 1	0 or 1	0 or 1	0 or 1	0 or 1	0-5
etc.	0 or 1	0 or 1	0 or 1	0 or 1	0 or 1	0-5
Total	(0-y)	(0-y)	(0-y)	(0-y)	(0-y)	0-5*y
Degree of agility (low level)	(0-y)/y	(0-y)/y	(0-y)/y	(0-y)/y	(0-y)/y	Total divided by number of cells in table

2.3. Applying the 4-DAT framework to existing agile methods

We can now apply this definition, in the form of the four dimensional framework of 4-DAT (Table 1), to an evaluation of existing (and proposed) software development methods – in this section, six well-known candidate agile methodologies. Such an evaluation can determine not only whether a method's elements can be considered agile or not (binary) but can determine the *degree* of agility exhibited by the method.

2.3.1. XP

The XP software development process focusses on iterative and rapid development. XP is characterized by six phases: Exploration, Planning, Iterations to first release, Productionizing, Maintenance and Death [15,16]. XP stresses communication and coordination among the team members at all times; and requires cooperation between the customer, management and development team to form the supportive business culture for the successful implementation of XP.

We have analyzed XP in detail from four different perspectives by using 4-DAT (Table 1). The first dimension of 4-DAT has been used to evaluate (qualitatively) the agile method XP in terms of its scope and usability. The results of the evaluation are presented in Table 3. This table shows that XP is suitable for small and medium sized projects, uses an iterative and incremental object-oriented approach to develop software products with a team size of less than 10. XP discusses code style (clean and simple), technology environment (requires quick feedback), physical environment (co-located and distributed teams) and business culture (collaborative and cooperative). We can therefore conclude that if there is a small or medium level project in-hand where the project requirements are not fixed then we may utilize XP for the development.

The second dimension has been used to evaluate XP from an agility perspective and the degrees of agility

have been measured in terms of the five variables (features) relating to Dimension 2 (see Table 1): flexibility (FY), speed (SD), leanness (LS), learning (LG) and responsiveness (RS) that may exist at some specific level or lifecycle phase or as a result of the practices used in the phases of XP. If any phase or practice of XP supports a particular agility feature, then 1 point is allocated in that particular cell, otherwise 0; and so on. [The detailed results of this calculation are given in Table 4]. The agile methods literature was used as the source of the numerical inputs for the analysis, calculated using the following equation [3]:

$$DA(Object) = (1/m) \sum m DA(Object, Phase \text{ or } Practices)$$

(for full discussion of this equation, see Appendix A). The cell values for XP (Table 4) for the five characteristics of Dimension 2 are for both the high level (phases) and low level (practices). For example, the planning phase in XP is not fixed and can be changed as we proceed in the software development; so it can be said to be a flexible phase – it is marked as 1 in the FY cell. Planning is done quickly for each release and iteratively; thus it can be marked as speedy and allotted 1 in the SD cell. The plan may change at any time, bringing an extra cost for fixing it; hence it is not lean and is consequently given a 0 in the LS cell. However, a plan can be considered as emergent in XP, with learning from previous releases, and is thus marked 1 in the LG cell. Finally, a plan responds whenever required and is thus marked 1 in the RS cell. The rest of the phases and practices in XP (and other agile methods) have been evaluated in a similar way. Table 4 shows that for XP the degree of agility is high in all categories except leanness (LS). For XP, the only “poor” phase is that of Death, with Maintenance having the second lowest value. Perhaps the most surprising results for XP are seen in Table 4 under Practices where it is clear that both “40-h week” and “metaphor” score badly – surprising because these are two of the practices most heavily stressed by XP advocates.

Table 3
Scope evaluation of six agile processes (XP, Scrum, FDD, ASD, DSDM and Crystal)

Criteria	XP	Scrum	FDD	ASD	DSDM	Crystal
Scope						
Project size	Small, medium	Small, medium, and scalable for large	Small, medium, and large (business projects/applications)	Large and Complex projects	Small and large projects (Business Applications)	Small and medium
Team size	<10	<10 and multiple teams*	No limit – scalable from small to large teams	Not mentioned	Minimum 2 and Maximum 6 (Multiple teams)	Single team in crystal clear with maximum 6 people in a team. Multiple teams with maximum 40 persons in orange and 80 persons in red methodology
Development style	Iterative, rapid	Iterative, rapid	Iterative design and construction	Iterative and rapid development – distributed development	Iterative, rapid development and cooperative	Iterative and rapid development
Code style	Clean and simple	Not specified	Not specified	Not mentioned	Not mentioned	Not mentioned
Technology environment	Quick feedback required	Not specified	Not specified	Not mentioned	Not mentioned	Not mentioned
Physical environment	Co-located teams and distributed teams (limited-interaction)	Not specified	Not specified	Co-located and distributed teams	Not mentioned	Co-located team – no support for distributed development
Business culture	Collaborative and cooperative	Not specified	Not specified	Not specified	Collaborative and cooperative	Not mentioned
Abstraction mechanism	Object-oriented	Object-oriented	Object-oriented	Object-oriented/Component-oriented	Object-oriented/Component-oriented	Object-oriented

*This is a special feature.

The third dimension of 4-DAT has been used to examine (qualitatively) the support of agile values in the different practices of XP (see Table 5). Here, we can see that XP offers support for all the initial agile values but the two most recently identified [3] are less well supported. XP offers no support for either “Keeping the process agile” – which might suggest a missing link to the SPI (software process improvement) community or for “Keeping the process cost effective” (a pragmatic agility value for commercial adoptability).

The fourth dimension of 4-DAT has been used to examine the practices of XP for the support of software processes. Table 6 presents the evaluation report for this fourth dimension. As with Dimension 3, the comparison is purely qualitative and informative (descriptive rather than prescriptive). Here, we can see that XP offers practices for the Development and Project Processes but says nothing about Configuration Management or Process Management.

2.3.2. Scrum

Schwaber and Beedle [17] report that Scrum is a flexible, adaptable, empirical, productive and iterative method that uses the ideas of industrial process control

theory for the development of software systems. According to [17], Scrum has three phases: Pre-Game, Development and Post-Game. Pre-Game phase has a further two sub-phases: planning and high level design (architecture).

Scrum is the second agile method that we have analyzed here and the detailed evaluation results are presented in Tables 3, 5–7. The results of scope evaluation (Table 3) of Scrum show that Scrum (an iterative and incremental object-oriented approach) is suitable for small, medium and large sized projects where team size is less than 10 people but where there may be multiple teams on the same project. Scrum does not explicitly discuss code style, technology environment, physical environment and business culture.

The degree of agility in Scrum has been measured in terms of the five variables (features) relating to Dimension 2 (see Table 1). If any phase or practice of Scrum supports a particular agility feature, then 1 point is allocated in that particular cell, otherwise 0; and so on as before – see Table 7 for details showing that the degree of agility in the “Post-Game” phase is very low in comparison with other phases. It also shows a zero score for leanness (LS) in all phases and practices of Scrum.

Table 4
Degree of agility in XP(after [5])

	Agility features					
XP	FY	SD	LS	LG	RS	Total
<i>Phases</i>						
Exploration	1	1	0	1	1	4
Planning	1	1	0	1	1	4
Iteration to release	1	1	0	1	1	4
Productionizing	1	1	1	1	1	5
Maintenance	1	0	0	1	1	3
Death	0	1	0	0	0	1
Total	5	5	1	5	5	21
Degree of Agility	5/6	5/6	1/6	5/6	5/6	21/(6*5)
<i>Practices</i>						
The planning game	1	1	0	1	1	4
Short release	1	1		1	1	4
Metaphor	0	1	1	0	0	2
Simple design	1	1	1	1	1	5
Testing	1	1	0	1	1	4
Refactoring	1	1	1	1	1	5
Pair programming	1	0	0	1	1	3
Collective ownership	1	0	0	1	1	3
Continuous integration	1	1	1	1	1	5
40-h week	0	0	0	1	0	1
On-site customer	1	0	0	1	1	3
Coding standards	1	1	1	1	1	5
Total	10	8	5	11	10	44
Degree of agility	10/12	8/12	5/12	11/12	10/12	44/(12*5)

Scrum practices have also been examined to check the support of agile values (see Table 5). We can see that Scrum offers support for all the initial five agile values but no support for the most recently identified [3] agile value “Keeping the process cost effective”. As we have already seen, none of Scrum’s phases or practices support leanness (LS) where the leanness attribute describes cost effectiveness (see Table 1).

Finally, we examined the practices of Scrum for the support of software processes. Table 6 clearly shows that Scrum offers practices for the Development and Project Management Processes but says nothing about Configuration management or Process management.

2.3.3. FDD

Feature Driven Development (FDD) focuses on design and planning, which differentiates it from other development approaches [14]. FDD provides guidelines, tasks, techniques and five sequential processes: Develop an Overall Model, Build a Feature List, Plan by Feature, Design by Feature and Build by Feature [18]. FDD software development focuses on iterative design and construction of the software product [18].

FDD is the third agile method that we have analyzed here and the detailed evaluation results are presented in Tables 3, 5, 6, and 8. The scope evaluation (Table 3) of FDD shows that FDD is an object-oriented approach that is suitable for the development (iterative design and construction) of small, medium and large sized projects where there is no limit on team size. It can be seen that FDD does

not explicitly specify code style, technology environment, physical environment and business culture, which is very similar to Scrum.

The degree of agility in FDD has been measured (for details see Table 8). It can be seen that the phases and practices of FDD support all agility attributes except Leanness (LS). It can also be seen that two FDD phases (“(1) Build by Feature” and “(2) Plan by Feature”) and one practice (“Developing by Feature”) do not support any of the agility attributes, which is quite surprising.

FDD practices have been examined for their support of agile values (see Table 5). The results show that FDD supports all the initial five agile values but does not support the sixth agile value of “Keeping the process cost effective”. FDD’s phases or practices do not support leanness (LS), which confirms that FDD lacks support for the agile value “Keeping the process cost effective”.

The support of software processes in FDD has been presented in Table 6, which clearly shows that FDD offers practices for the Development, Project Management Processes and Configuration Management but says nothing about Process Management.

2.3.4. ASD

Adaptive Software Development (ASD) provides a framework that encompasses concepts, guidelines and practices in order to encourage rapid, incremental and iterative development of complex systems with constant prototyping. The Adaptive Development Model is characterized by three phases: Speculate, Collaborate and Learn [19]. ASD addresses issues such as social, cultural and team skills that are aligned with distributed software development and also provides techniques to deal with them.

According to the scope evaluation (see Table 3), ASD (an object-oriented approach) facilitates iterative and rapid delivery of large complex systems where ASD project teams (no limit for team size) may be co-located or distributed at different geographical locations. It is clear from this analysis that ASD addresses issues such as social, cultural and team skills that are aligned with distributed software development but does not explicitly say anything about code style, technology environment or business culture.

Table 9 presents the measured degrees of agility in ASD both at process and practices level. It can be seen here that the phases and practices of ASD support all agility attributes (like other agile methods) except Leanness (LS). It is also noted that the ASD phase (“The Project Initiation”) and practice (“The Project Mission Development”) do not support any of the agility attributes and are therefore marked “zero” in the cells.

According to our analysis, ASD (see Table 5) supports all the initial five agile values but does not offer support for the recently identified sixth agile value of “Keeping the process cost effective”. As we have already seen, ASD’s phases or practices do not support leanness (LS), which also offers support for the observation that ASD does

Table 5
Degree of agility for Dimension 3 in six agile methods (XP, Scrum, FDD, ASD, DSDM and Crystal)

Agile values	XP	Scrum	FDD	ASD	DSDM	Crystal
Individuals and interactions over processes and tools	1. The planning game 2. Collective ownership 3. On-site customer 4. Pair programming	1. Scrum teams 2. Sprint planning meeting 3. Daily scrum meeting	1. Domain object modelling 2. Individual class ownership 3. Feature teams 4. Inspection	1. Adaptive management model 2. Collaborative teams 3. Joint application development by independent agents 4. Customer focus group reviews	1. Active user involvement 2. Empowered teams. 3. Collaboration and cooperation among stakeholders	1. Holistic Diversity and Strategy 2. Parallelism and Flux 3. User Viewings
Working software over comprehensive documentation	1. Short releases 2. Testing 3. Continuous integration	1. Sprint 2. Sprint review	1. Developing by Feature 2. Inspection 3. Regular Builds 4. Reporting/visibility of results	1. Developing by components 2. Software inspection 3. Project postmortem	1. Frequent product delivery 2. Iterative and incremental development 3. Integrated testing	1. Revision and Review 2. Monitoring of a Progress
Customer collaboration over contract negotiation	1. The planning game 2. On-site customer	1. Product backlog 2. Sprint planning meeting	1. Domain object modelling.	1. Adaptive management model 2. Joint application development	1. Collaboration and cooperation among stakeholders 2. Requirements are baselined at a high level.	1. Staging 2. User Viewings.
Responding to change over following a plan	1. Metaphor 2. Simple Design 3. Refactoring 4. Coding standards	1. Sprint review 2. Sprint planning meeting	1. Domain object modelling 2. Configuration Management	1. Adaptive cycle planning 2. Customer focus group reviews	1. Reversible changes	1. Reflection Workshops 2. Methodology Tuning
Keeping the process agile	–	1. Sprint review 2. Daily scrum meeting	1. Reporting/visibility of results 2. Inspection	1. Software inspection 2. Project Postmortem	1. Integrated testing	1. Reflection Workshops 2. Monitoring of a Progress
Keeping the process cost effective	–	–	–	–	–	1. Reflection Workshops 2. Monitoring of Progress

not support the agile value “Keeping the process cost effective”.

Finally, further analysis results in Table 6 show that ASD offers practices for the Development Process, Project Management Processes, Process Management Process but has no such practices for Configuration Management.

2.3.5. DSDM

The Dynamic Software Development Method [20,21] provides a framework (rather than a method) that supports rapid, iterative and collaborative software development for producing high quality business information systems solutions [22]. The basic principle of DSDM is that the resources and timeframe are adjusted and then the goals

and the required functionality (that is not fixed) are adjusted accordingly [23]. According to the DSDM Consortium [24], a DSDM project has seven phases: Pre-Project, Feasibility Study, Business Study, Functional Model Iteration, Design and Build Iteration, Implementation and Post-Project.

According to the scope evaluation (see Table 3), DSDM (object-oriented/component-oriented approach) is suitable for the iterative and rapid development of small to large business applications. A DSDM project may have multiple teams where team size may vary from 2 members (minimum) to 6 members (maximum). It can also be seen that the business culture in DSDM projects is collaborative and cooperative and that there is no fixed contract between

Table 6

Software process in six agile methods (XP, , Scrum, FDD, ASD, DSDM and Crystal) (Dimension 4)

Software process	XP	Scrum	FDD	ASD	DSDM	Crystal
Development process	1. Short releases 2. Metaphor 3. Simple design 4. Testing 5. Refactoring 6. Pair programming 7. Collective Ownership 8. Continuous integration 9. On-site customer 10. Coding standard	1. Scrum teams 2. Product backlog 3. Sprint 4. Sprint review	1. Domain object modeling 2. Developing by feature 3. Individual class ownership 4. Feature teams 5. Inspection 6. Regular builds	1. The project mission development 2. Developing by components. 3. Collaborative teams 4. Joint application development 5. Customer focus group reviews 6. Software inspection	1. Active user involvement 2. Empowered teams 3. Frequent product delivery 4. Fitness for business purpose 5. Iterative and incremental development 6. Reversible changes 7. Requirements are baselined at high level 8. Integrated testing 9. Collaboration and cooperation among stakeholders	1. Staging 2. Holistic Diversity and Strategy 3. Parallelism and Flux 4. User Viewings. 5. Revision and review
Project management process	1. The planning game	1. Scrum master 2. Sprint planning meeting 3. Daily scrum meeting	1. Reporting/visibility of results	1. Adaptive cycle planning 2. Adaptive management model	Not specified	1. Monitoring of a progress
Software configuration control process/support process	Not specified	Not specified	1. Configuration management	Not specified	Not specified	Not specified
Process management process	Not specified	Not specified	Not specified	1. Project postmortem	Not specified	1. Reflection workshops Methodology tuning

the development company and the customer. DSDM does not specify code style, technology environment and physical environment.

The degree of agility in DSDM can be seen in Table 10. Like previously evaluated agile methods, here we can also see that DSDM, both at the process and practices level, supports all agility attributes except Leanness (LS). It is also noted that three DSDM phases (“The Pre-Project”, “Feasibility Study” and “Business Study”) and one practice (“Requirements are baselined at High Level”) do not support any of the agility attributes at all and are marked as having “zero” agility in the cells.

DSDM (see Table 5) supports all the initial five agile values but does not offer support for the recently identified sixth agile value, “Keeping the process cost effective”. As

with the other assessed agile methods, this is because DSDM’s phases or practices do not support leanness (LS), which is related to the cost and economic perspective of a software development process.

Finally, Table 6 shows that DSDM offers practices for the Development Process but does not say anything about Project Management Processes, Process Management Process or Configuration Management.

2.3.6. Crystal

Crystal methodologies focus on incremental development (which may be in parallel), where one increment may take several iterations to complete. The tunable project life cycle that is common for all Crystal methodologies is: envisioning, proposal, sales, setup, requirements, design and code, test, deploy, train, alter [25]. According to [22],

Table 7
Degree of agility in Scrum – cell values (after [5])

	Agility features					
Scrum	FY	SD	LS	LG	RS	Total
<i>Phases</i>						
Pre-Game	1	1	0	1	1	4
Development	1	1	0	1	1	4
Post-Game	0	1	0	0	0	1
Total	2	3	0	2	2	9
Degree of agility	2/3	3/3	0/3	2/3	2/3	9/(3*5)
<i>Practices</i>						
Scrum master	1	1	0	1	1	4
Scrum teams	1	1	0	1	1	4
Product backlog	1	1	0	1	1	4
Sprint	1	1	0	1	1	4
Sprint planning meeting	1	1	0	1	1	4
Daily scrum meeting	1	1	0	1	1	4
Sprint review	1	1	0	1	1	4
Total	7	7	0	7	7	28
Degree of agility	7/7	7/7	0/7	7/7	7/7	28/(7*5)

Table 8
Degree of agility in FDD – cell values

	Agility features					
FDD	FY	SD	LS	LG	RS	Total
<i>Phases</i>						
Develop an overall model	1	1	0	1	1	4
Build a feature list	0	0	0	0	0	0
Plan by feature	0	0	0	0	0	0
Design by feature (per-feature)	1	1	0	1	1	4
Build by feature (per-feature)	1	1	0	1	1	4
Total	3	3	0	3	3	12
Degree of agility	3/5	3/5	0/5	3/5	3/5	12/(5*5)
<i>Practices</i>						
Domain object modelling	1	1	0	1	1	4
Developing by feature	0	0	0	0	0	0
Individual class ownership	1	1	0	1	1	4
Feature teams	1	1	0	1	1	4
Inspection	1	1	0	1	1	4
Regular builds	1	1		1	1	4
Configuration management	1	1	0	1	1	4
Reporting/visibility of results	1	1	0	1	1	4
Total	7	7	0	7	7	28
Degree of agility	7/8	7/8	0/7	7/8	7/8	28/(8*5)

Crystal Clear has one team whereas a Crystal Orange team may be split into several cross-functional groups [6] by using the Holistic Diversity strategy [26]. The development process is carried out by following guidelines of policy standards, work products, local matters, tools, standards and roles, which are all provided by the Crystal family of methodologies [6]. According to [27], the Crystal family of methodologies are people-centred and share a human orientation with XP where people continuously monitor their process and tune it as they make progress.

Finally, here we have analyzed, as the sixth agile software development approach, the Crystal family of methodologies. According to the scope evaluation (see Table 3), Crystal (object-oriented approach) offers different methodologies (named as Crystal Clear, Orange etc.) for the itera-

Table 9
Degree of agility in ASD – cell values

ASD	Agility features					Total
	FY	SD	LS	LG	RS	
<i>Phases</i>						
The project initiation	0	0	0	0	0	0
Adaptive cycle planning	1	1	0	1	1	4
Concurrent component engineering	1	1	0	1	1	4
Quality review	1	1	0	1	1	4
Final Q/A and release	0	1	0	1	0	2
Total	3	4	0	4	3	14
Degree of agility	3/5	4/5	0/5	4/5	3/5	14/(5*5)
<i>Practices</i>						
The project mission development	0	0	0	0	0	0
Adaptive cycle planning	1	1	0	1	1	4
Developing by components	1	1	0	1	1	4
Adaptive management model	1	1	0	1	1	4
Collaborative teams	1	1	0	1	1	4
Joint application development by independent agents	1	1	0	1	1	4
Customer focus group reviews	1	1	0	1	1	4
Software inspection	1	1	0	1	1	4
Project postmortem	0	1	0	1	0	2
Total	7	8	0	8	7	30
Degree of agility	7/9	8/9	0/9	8/9	7/9	30/(9*5)

tive and rapid development of small to large applications. A Crystal project may have multiple teams where a single team may have up to 6 members (maximum). It can be seen that Crystal does not support a distributed development; therefore development teams should be co-located. Crystal does not specify code style, business culture and technology environment explicitly.

Table 11 presents the degree of agility in Crystal. Here we can see that at the process level (Phases), Crystal supports all agility attributes except Leanness (LS) but at the practice level there are two practices (“Reflection Workshops” and “Monitoring of a Progress”) that support leanness (LS), which makes it distinct from the other agile methods evaluated in this paper. It is also noted that there is only one practice (“Holistic Diversity and Strategy”) with a minimum degree of agility as compared to other practices in Crystal.

Crystal practices (see Table 5) support all the initial five agile values and also support the recently identified sixth agile value “Keeping the process cost effective”, unlike other assessed agile methods where poor support was seen.

Finally, Table 6 shows that Crystal offers practices for the Development Process, Project Management Process and Process Management Process but does not say anything about Configuration Management.

2.3.7. Inter-method comparison

In the last few sections, we have individually presented and analyzed six agile methods. Here, in this section, we present an inter-method comparison that first outlines the high level differences between these methods and their possible usability in different projects; and then ranks them

Table 10
Degree of agility in DSDM – cell values

	Agility features					
DSDM	FY	SD	LS	LG	RS	Total
<i>Phases</i>						
Pre-project	0	0	0	0	0	0
Feasibility study	0	0	0	0	0	0
Business study	0	0	0	0	0	0
Functional model iteration	1	1	0	1	1	4
Design and build iteration	1	1	0	1	1	4
Implementation	1	1	0	1	1	4
Post-project	1	1	0	1	1	4
Total	4	4	0	4	4	16
Degree of agility	4/7	4/7	0/7	4/7	4/7	16/(7*5)
<i>Practices</i>						
Active user involvement	0	1	0	1	1	3
Empowered teams	1	1	0	1	1	4
Frequent product delivery	1	1	0	1	1	4
Iterative and incremental development	1	1	0	1	1	4
Reversible changes	1	1	0	1	1	4
Requirements are baselined at High Level	0	0	0	0	0	0
Integrated testing	1	1	0	1	1	4
Collaboration and cooperation among stakeholders.	1	1	0	1	1	4
Total	6	7	0	7	7	27
Degree of agility	6/8	7/8	0/8	7/8	7/8	27/(8*5)

according to their degree of agility. It is clear from the above analysis that XP and Scrum are suitable for small and medium projects; Crystal, FDD and DSDM are suitable for small, medium and large business systems; and ASD is suitable for large and complex projects. Team size is not specified by FDD and ASD whereas XP, DSDM, Scrum and Crystal specify the team size. All methods produce software in rapid increments but ASD also discusses distributed software development. Only XP discusses code style (clean and simple); others do not specify this explicitly. XP, ASD and Crystal discuss the physical environment for development, others remaining silent. XP and DSDM are the only methodologies to explicitly specify a collaborative and cooperative business culture for software development. Furthermore, we have used the Dimension 2 (Table 1) and a template (Table 2) to measure the degree of agility in six selected agile methods. Table 12 presents the final value for the degree of agility in these six methods.

We can see (Table 12 and Fig. 1) that Crystal is evaluated as the most agile at the phase level and Scrum shows itself to be the most agile at the practices level. Furthermore, DSDM is assessed as being less agile at the phase level and ASD as less agile at the practices level in comparison with other agile methods. Table 12 also shows the ranking of agile methods and Fig. 1 summarizes these results graphically for the degree of agility in selected agile methods.

Application of the 4-DAT approach can thus give methodologists and managers in an organization some quantitative as well as qualitative data on how agile any method is. For example, an organization may have decided to adopt an agile approach and is unsure which of the many possibilities to choose. As well as giving an

overall “picture” of a method, the more detailed values, at both process and practices level, may be useful when specific characteristics of an agile method are needed by the organization or on a specific project. Perhaps it is important to have a very lean project but the speed characteristic is of less importance in this particular case. The 4-DAT approach thus provides additional, quasi-objective insights to help the project manager make the best method choice.

3. Agility within traditional methods

Although the 4-DAT was developed as an assessment tool for agile methods, it is interesting here, as a test of its utility, to apply it to two traditional software development methods: waterfall and spiral. For 4-DAT to be acceptable, one would conjecture that these calculations should lead to significantly lower values than for those agile methods of Section 2.

3.1. Water fall software development process

The ‘waterfall’ approach [28] was the very first formal software development process. It is still in use and is most applicable where project requirements are fixed. The waterfall approach may be characterized by three main phases: Project Initiation, Development and Testing. Waterfall consists of five main practices: System Concept, Analysis, Design, Coding and Testing. Although waterfall has been used for a long time, its current incarnations tend not to follow the original publications but to be some more sophisticated variant of it.

Table 11
Degree of agility in Crystal – cell values

	Agility features					
CRYSTAL	FY	SD	LS	LG	RS	Total
<i>Phases</i>						
Planning	1	1	0	1	1	4
Development phase for increment	1	1	0	1	1	4
User-useable release	1	1	0	1	1	4
Total	3	3	0	3	3	12
Degree of agility	3/3	3/3	0/3	3/3	3/3	12/(3*5)
<i>Practices</i>						
Staging	1	1	0	1	1	4
Reflection workshops	1	1	1	1	1	5
Monitoring of a progress	0	1	1	0	1	3
Methodology tuning	1	1	0	1	1	4
Holistic diversity and strategy	0	1	0	0	1	2
Parallelism and flux	1	1	0	0	1	3
User viewings	1	1	0	1	1	4
Revision and review	1	1	0	1	1	4
Total	6	8	0	5	8	29
Degree of agility	6/8	8/8	2/8	5/8	8/8	29/(8*5)

According to our scope evaluation analysis (see Table 13), the waterfall approach supports a linear or sequential style of development and can be used for small, medium or large complex systems where the project requirements and contract (with a customer) are fixed. It does not specify team size, code style, physical environment or abstraction mechanism.

Table 14 presents the measured degrees of agility in the waterfall approach, both at the process and practices level. It can be seen that the phases and practices of waterfall do not support any of the agility attributes and therefore are marked “Zero” in the cell. This is due to inflexible and rigid nature of the waterfall approach. However, waterfall can be considered as a generic theoretical model for many subsequent software processes.

According to our analysis (see Table 15), waterfall does not support any of the agile values. As we have already seen, the phases and practices of the waterfall approach do not support any of the agility attributes, which also offers support for the observation that waterfall does not support the agile values either.

Finally, further analysis results in Table 16 show that waterfall offers practices for the Development Process, Project Management Processes, but does not explicitly specify practices for Configuration Management and Process Management.

3.2. Spiral model for software development process

The Spiral Model for software development [29] is also known as risk-driven development. It includes practices from waterfall, prototyping and incremental models for software processes. The spiral model may be characterized by five main phases: Project Concept, Risk Management, Planning, Prototyping, and Product Engineering. The spiral model consists of ten main practices: Project Concept, Risk Management, Prototype Planning, Prototyping,

Concepts of Operations, Software Requirements, Development Plan, Design, Coding and Testing. This approach is often characterized as a set of four development cycles [although it has also been observed that linearity is a more dominant feature than iteration [30]]. In the first “cycle”, initial risk analysis, initial prototype, initial requirement plan and project concepts are undertaken. In the second “cycle”, risk analysis, detailed prototype, software requirements, requirements validation and development plan are the focus. In the third “cycle”, risk analysis, next level prototype, initial design, design validation, integration and test plan are documented. In the fourth “cycle”, operational prototype, risk analysis, detailed design, implementation and testing are done. We may apply separate spiral models (a set of cycles) for the incremental development of separate software modules or components [29].

It has been argued that the spiral model combines the strengths of waterfall, prototyping and an incremental model. Another important aspect of a spiral model is the consideration of risk assessment and management. Although the spiral model “cycles” focus on the iterative risk assessment and the development of prototypes, each “cycle” does not deliver executable code, whereas each iteration in an agile software development does in fact deliver such executable code. In each “cycle” of the spiral, full planning (fixed planning) is undertaken for the next “cycle”, which would appear to support incremental planning rather than an emerging or evolving plan. For example, a fixed plan is developed for requirements in the first “cycle”, then a fixed plan is done for development in the second “cycle”, and then a fixed plan is done for testing. In a similar way, first a preliminary design is done followed by a detailed design (iterative design) is done. The only evolving and emerging practices of spiral are risk management, prototype planning and prototype development that evolve as the spiral moves on for the next “cycle”. Never-

Table 12
Degree of agility in XP, Scrum, FDD, ASD, DSDM, Crystal – overall comparison

Process & practices	XP	Scrum	FDD	ASD	DSDM	Crystal
Phases	21/30 = 0.70	9/15 = 0.60	12/25 = 0.48	14/25 = 0.56	16/35 = 0.46	12/15 = 0.80
Rank	2	3	5	4	6	1
Practices	44/60 = 0.73	28/35 = 0.80	28/40 = 0.70	30/45 = 0.67	27/40 = 0.68	29/40 = 0.73
Rank	2	1	3	5	4	2

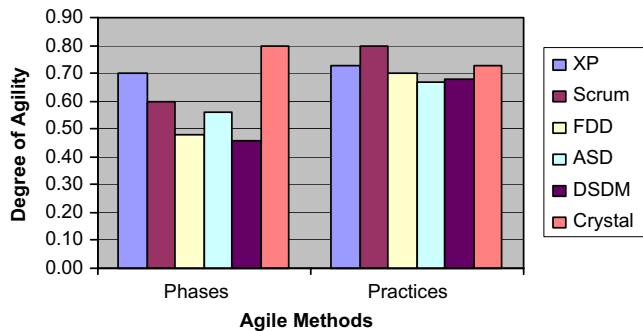


Fig. 1. Comparison of the degree of agility for six agile methods as measured for the phases level and the practices level.

theless, all the artifacts and practices are documentation-oriented (detailed documentation) and the fact of face-to-face communication is not considered as an important activity whereas agile methods focus on face-to-face communication with reduced documentation. However, we can note that the only incremental aspect of a spiral model is similar to agile.

According to our scope evaluation (see Table 13), the spiral model follows a hybrid development style (sequential and incremental) and is suitable for medium and large projects where the project requirements and customer contract are fixed. It does not specify the team size, code style, physical environment or abstraction mechanism.

Table 17 presents the measured degrees of agility in the spiral model at both the process and practices level. It can be seen that the ‘Risk Management’ and ‘Prototyping’ Phases are flexible, iterative and evolving in nature. They therefore respond to changes, both showing a high degree of agility in comparison with other phases. At the practices level, it is also clear that only three practices encompass a high degree of agility and the rest of the practices show

poor support for agility attributes. Although spiral delivers the software product in small increments, which shows support for “SD”, it fails to support the “LS” factor since spiral is a documentation-intensive software process that prevents it achieving a high value for “LS”. In a similar way, we have calculated the degree of agility in other spiral model phases and practices.

According to our analysis, three of the practices (see Table 15) support only one agile value “responding to change over following a plan” and the rest do not support any of the agile values. As we have already seen, Risk Management, Prototype Planning and Prototype Development practices show a significant degree of agility, which also offers support for the observation that the rest of the spiral model practices do not support the agile values.

Finally, further analysis results in Table 16 show that a spiral model offers practices for the Development Process, Project Management Process, Risk Management Process but does not explicitly specify practices for Configuration Management and Process Management.

3.3. Brief Inter-method comparison

To compare the two studied traditional software development approaches with agile methodologies, we have added the summary data from Tables 14 and 17 to Fig. 1, grouping the Phases and Practices values together for each methodology, thus giving a slightly different visual insight into this comparison (Fig. 2). It is immediately obvious that the two traditional (ostensibly non-agile) methodologies have values for both Phases and Practices well below the values for the agile methods analyzed in Section 2 and below the threshold of 0.5–0.6 proposed earlier. What is interesting, however, is that for Phases, the value for the spiral approach is not that much lower than for

Table 13
Scope evaluation of the waterfall and spiral approaches

Criteria	Waterfall	Spiral
<i>Scope</i>		
Project size	Small, medium, large	Medium, large
Team size	Does not specify	Does not specify
Development style	Linear and sequential	Hybrid (sequential, incremental)
Code style	Does not specify	Does not specify
Technology environment	Does not specify	Does not specify
Physical environment	Does not specify	Does not specify
Business culture	Fixed contract	Fixed contract
Abstraction mechanism	Does not specify	Does not specify

Table 14
Degree of agility in the waterfall approach – cell values

	Agility features					
Water fall	FY	SD	LS	LG	RS	Total
<i>Phases</i>						
Project initiation	0	0	0	0	0	0
Development	0	0	0	0	0	0
Testing	0	0	0	0	0	0
Total	0	0	0	0	0	0
Degree of agility	0/3	0/3	0/3	0/3	0/3	0/(3*5)
<i>Practices</i>						
System concept	0	0	0	0	0	0
Analysis	0	0	0	0	0	0
Design	0	0	0	0	0	0
Coding	0	0	0	0	0	0
Testing	0	0	0	0	0	0
Total	0	0	0	0	0	0
Degree of agility	0/5	0/5	0/5	0/5	0/5	0/(5*5)

Table 15
Degree of agility for Dimension 3 in the waterfall and spiral approaches

Agile values	Water fall	Spiral
Individuals and interactions over processes and tools	–	–
Working software over comprehensive documentation	–	–
Customer collaboration over contract negotiation	–	–
Responding to change over following a plan	–	Risk Management Prototype Planning prototyping
Keeping the process agile	–	–
Keeping the process cost effective	–	–

Table 16
Software process in the two traditional approaches

Software process	Waterfall	Spiral
Development process	Analysis Design Coding Testing	Project concept Operations concept Prototyping Software Requirements Design Coding Testing
Project management process	System concept	Prototype planning Development plan
Software configuration Control process/Support process	Not specified	Risk management
Process management process	Not specified	Not specified

DSDM. This perhaps reflects the fact that DSDM was not originally developed as an agile approach and has only recently been included in such lists of agile approaches. For the more detailed Practices evaluation, even the spiral approach comes nowhere near approaching even the lowest of the agile values.

Table 17
Degree of agility in the spiral approach – cell values

	Agility features					
Spiral	FY	SD	LS	LG	RS	Total
<i>Phases</i>						
Project concept	0	0	0	0	0	0
Risk management	1	1	0	1	1	4
Planning	0	1	0	0	0	1
Prototyping	1	1	0	1	1	4
Product engineering	0	1	0	0	0	1
Total	0	0	0	0	0	0
Degree of agility	2/5	4/5	0/5	1/5	2/5	10/(5*5)
<i>Practices</i>						
Project concept	0	0	0	0	0	0
Risk management	1	1	0	1	1	4
Prototype planning	1	1	0	1	1	4
Operation concepts	0	1	0	0	0	1
Prototyping	1	1	0	1	1	4
Requirements	0	1	0	0	0	1
Development plan	0	1	0	0	0	1
Design	0	1	0	0	0	1
Coding	0	1	0	0	0	1
Testing	0	1	0	0	0	1
Total	0	0	0	0	0	0
Degree of agility	3/10	9/10	0/10	3/10	3/10	18/(10*5)

4. Situational method engineering and the use of 4-DAT

As a further indication of the potential value of the 4-DAT, here we briefly overview how it may be useful in the context of situational method engineering (SME). In a SME approach [31–34], small pieces of a method are identified and stored as method fragments or method chunks [35] in a repository or methodbase [36,37]. For each project, the method engineer selects appropriate method fragments from the methodbase, perhaps with the help of an outside consultant or a software tool such as a Computer Assisted Method Engineering (CAME) tool [37–39]. The method is thus “constructed” or engineered from its component parts in such a way that only relevant process components, as represented by the method fragments, are incorporated into the constructed method and those not useful can be safely ignored e.g. [33]. Construction rules are required for this assembly process [40,41] – this is arguably one of the most challenging parts of SME. In the construction of an agile method, it is important to be able to identify those fragments that may have appropriate agility (and the extent of that agility characteristic).

In traditional SME, it is left to the judgement of the method engineer to ascertain whether the method fragment selected from the methodbase/repository is appropriate or not. For the construction of an agile methodology, we can use the 4-DAT to add an “agility value” to any such fragments i.e. it is applied at the high granularity level of a method fragment rather than at the methodology level as discussed in Section 2. Low values suggest that the fragment is of low agility value overall. Thus, such values can

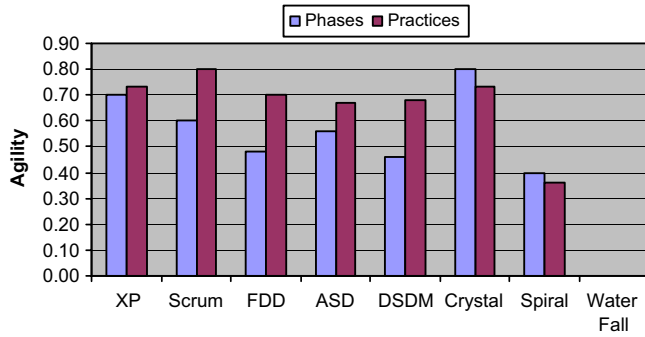


Fig. 2. Comparison of the degree of agility derived from 4-DAT evaluations for the two traditional methods (spiral and waterfall) as compared to values derived earlier for agile methods – this time grouped according to methodology rather than phase/practice.

assist method engineers in their decision-making process with respect to the degree of agility they seek in each method fragment.

In summary, the use of situational method engineering has three direct advantages to the software development organization and to the software development team:

- (i) The method that is constructed for the current project is *ultimately* what is required both in terms of processes, activities, tasks, techniques, guidelines (depending on your choice of terminology) as well as in terms of the people involved (actually roles), the extent of bureaucratic reporting and project management and the lifecycle model itself – and now the extent of agility implicit in the fragment.
- (ii) For subsequent projects, the team will develop confidence in self-tuning (tailoring) their own method to fit these projects and also in responding to any change in the development environment – perhaps the scope of the projects they undertake will change; as they grow in sophistication and software development capability so must the process (see also [42]). Consequently, elements in the method repository previously eschewed may now prove useful. Perhaps there is a technique for finding classes and agents that the project team felt they did not have the skills to use previously; perhaps there is a training role for team expansion.
- (iii) Building trust in the team members' ability to make use of their method. The development team will be highly motivated and enthusiastic to fully utilize their method as a result of gaining method ownership.

5. Conclusions

An analytical framework (4-DAT) has been developed and applied to the detailed evaluation of the degree of agility in six selected agile methods and two traditional methods (waterfall and spiral). The evaluation approach in this analytical framework evaluates these methods from four perspectives both at the process level and practice level.

Such an evaluation can be used to rationally select an appropriate agile method/process for a particular development or in method engineering, where a situation specific method/process can be constructed with an appropriate degree of agility.

Acknowledgements

We thank the anonymous reviewer who suggested comparing agile methodologies with traditional waterfall and spiral approaches (Section 3). We also wish to thank the Australian Research Council for financial support under their Discovery scheme. This is contribution number 07/04 of the Centre for Object Technology Applications and Research.

Appendix A. Details of the algorithms used to calculate agility in the 4-DAT

(a) equations used

The degree of agility (DA) is a value between 0 and 1 and is given by

$$DA(Object, Phase) = \{PA, FA\}$$

$$DA(Object, Practices) = \{PA, FA\}$$

where PA = partial agility and FA = full agility or

$$DA(Object) = (1/m) \sum m DA(Object, Phase \text{ or } Practices)$$

where $Object = \{Organization, System, Method, Process, Software, Documentation, Activities, Techniques, Metamodel, Method Engineering, Development, Engineering, \dots\}$ and Phase must be specified qualitatively.

For example, we could write the fact that for $Object = \text{Method}$, $LA = \text{Requirements Engineering}$, wherein the value of DA was classed as partial (PA) as:

$$DA(Object = Method, Phase = Requirements Engineering) = PA$$

A method (MD) may have process(es), whereby each process PS may have different phases. In one or more phase(s) PH, different practices PR may be applied in order to contribute towards the solution of a given problem. We can thus alternatively express the Process in terms of these cross-phase practices. The following equations summarize these concepts.

$$MD = Method = \{PS1, PS2, \dots, PSn\}$$

$$PS(PH) = Process \text{ based on phases} = \{PH1, PH2, \dots, PHn\}$$

$$PS(PR) = Process \text{ based on practices} = \{PR1, PR2, \dots, PRn\}$$

Agility in a process (phases): First the agility is calculated in terms of the five features of agility (see main text) for each phase of a process.

Agility of $PH_i = \{FY + SD + LS + LG + RS\}$

Total Agility in $PS = \text{Agility of } (PH_1 + PH_2 + \dots + PH_n)$

Degree of Agility in PS based on Phases, $PS(PR) =$
 $(\text{Total Agility in } PS) / (n * 5)$

where n is number of phases in a process and the constant 5 represents the five agility attributes.

Agility in a process (practices): The agility is calculated in terms of the five features of agility for each practice of a process.

Agility in $PR_i = \{FY + SD + LS + LG + RS\}$

Total Agility in $PS = \text{Agility of } (PR_1 + PR_2 + \dots + PR_n)$

Degree of Agility in PS based on Practices, $PS(PR) =$
 $(\text{Total Agility in } PS) / (n * 5)$

References

- [1] T. Chau, F. Maurer, G. Melnik, Knowledge sharing: agile methods vs. Tayloristic methods, *Procs. 12th IEEE International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE'03)*, IEEE Computer Society Press, Los Alamitos, CA, USA, 2003, pp. 302–307.
- [2] AgileManifesto. Manifesto for Agile Software Development, <http://www.agilemanifesto.org/>. 2001, Accessed 14 March 2005.
- [3] A. Qumer, B. Henderson-Sellers, Measuring agility and adoptability of agile methods: a 4-dimensional analytical tool, *Procs. IADIS International Conference Applied Computing 2006* (eds. N. Guimarães, P. Isaias and A. Goikoetxea), IADIS Press, 2006, pp. 503–507.
- [4] A. Qumer, B. Henderson-Sellers, Crystallization of agility: back to basics, *Proceedings of the First International Conference on Software and Data Technologies*, INSTICC Press, Volume 2, 2006, pp. 121–126.
- [5] A. Qumer, B. Henderson-Sellers, Comparative evaluation of XP and Scrum using the 4D Analytical Tool (4-DAT), in: Z. Irani, O.D. Sarikas, J. Llopis, R. Gonzalez, J. Gasco (Eds.), *Proceedings of the European and Mediterranean Conference on Information Systems 2006 (EMCIS2006) CD*, Brunel University, West London, 2006.
- [6] A. Cockburn, *Agile Software Development*, Addison-Wesley, Boston, 2002.
- [7] R. Dove, The Meaning of Life and the Meaning of Agility. Paradigm Shift International www.parshift.com/library.htm, 1997.
- [8] B.A. Kitchenham, L. Jones, Evaluating Software Engineering Methods and Tool part 5: the Influence of Human Factors, *ACM SIGSOFT Software Engineering Notes* 22 (1) (1997) 13–15.
- [9] L. Williams, W. Krebs, L. Layman, A.I. Anton, P. Abrahamsson, Toward a Framework for Evaluating Extreme Programming. Empirical Assessment in Software Engineering, Edinburgh, 2004, pp. 11–20.
- [10] B. Boehm, R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*, Pearson Education, Inc., Boston, 2004.
- [11] P. Cuesta, A. Gómez, J.C. González-Moreno, F.J. Rodríguez, A Framework for Evaluation of Agent Oriented Methodologies. Workshop on Agent-Oriented Information Systems (AOIS-2002), 2002.
- [12] Q.-N.N. Tran, G. Low, M.-A. Williams, A Preliminary Comparative Feature Analysis of Multi-agent Systems Development Methodologies, in: 6th International Bi-Conference Workshop on Agent-Oriented Information Systems, Springer-Verlag, New York, USA, 2004, pp. 386–398.
- [13] P. Jalote, *An Integrated Approach to Software Engineering*, Springer-Verlag, New York, 1997.
- [14] A.S. Koch, *Agile Software Development: Evaluating the Methods for Your Organization*, Artech House, Inc., London, 2005.
- [15] K. Beck, *Extreme Programming Explained*, Addison-Wesley, Pearson Education, 2000.
- [16] R.G. Wolak, System Development: Research Paper 1, SDLC on a Diet. <http://www.itstudyguide.com/papers/rwDISS725researchpaper1.htm>, 2001.
- [17] K. Schwaber, M. Beedle, *Agile Software Development with SCRUM*, Prentice Hall, 2002.
- [18] S.R. Palmer, J.M. Felsing, *A Practical Guide to Feature-Driven Development*, Prentice-Hall Inc, Upper Saddle River, 2002.
- [19] J.A.I. Highsmith, *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems*, Dorset House Publishing, New York, 2000.
- [20] J. Stapleton, *DSDM: The Method in Practice*, Addison-Wesley, Reading, MA, 1997.
- [21] Wikipedia., *Agile Software Development*. The Wikimedia Foundation, Inc., <http://en.wikipedia.org>, 2005.
- [22] P. Abrahamsson, O. Salo, J. Ronkainen, J. Warsta, Agile software development methods. Review and analysis Report. VTT Publications 478, 2002.
- [23] P. Abrahamsson, J. Wasta, M.T. Siponen, J. Ronkainen, New Direction on agile Methods: a Comparative Analysis. 25th International Conference on Software Engineering. IEEE Computer Society, Portland Oregon, 2003.
- [24] DSDM., DSDM Consortium. Dynamic Systems Development Method Ltd., <http://www.dsdm.org>, 2003.
- [25] A. Cockburn, A Methodology Per Project, <http://alistair.cockburn.us/crystal/articles/mpp/methodologyperproject.html>, 1999.
- [26] A. Cockburn, *Surviving Object-Oriented Projects: A Manager's Guide*, Addison Wesley Longman, 1998.
- [27] M. Fowler, The New Methodology, <http://www.martinfowler.com/articles/newMethodology.html>, 2003.
- [28] Royce, W. Winston W. Royce. Managing the Development of Large Software Systems: Concepts and Techniques. In *WESCON Technical Papers*, Western Electronic Show and Convention, Los Angeles, Aug. 25–28, number 14. 1970. Reprinted in *Proceedings of the Ninth International Conference on Software Engineering*, Pittsburgh, PA, USA, ACM Press, 1989, pp. 328–338.
- [29] B.W. Boehm, A Spiral Model of Software Development and Enhancement. *Computer IEEE JNL* 21 (5) (1988) 61–72.
- [30] B. Henderson-Sellers, J.M. Edwards, *BOOK TWO of Object-Oriented Knowledge: The Working Object*, Prentice-Hall, 1994 (594pp).
- [31] K. Kumar, R.J. Welke, Methodology engineering: a proposal for situation-specific methodology construction, in: W.W. Cotterman, J.A. Senn (Eds.), *Challenges and Strategies for Research in Systems Development*, J. Wiley, Chichester, 1992, pp. 257–269.
- [32] S. Brinkkemper, Method engineering: engineering of information systems development methods and tools, *Inf. Software Technol* 38 (4) (1996) 275–280.
- [33] S. Brinkkemper, M. Saeki, F. Harmsen, Meta-modelling based assembly techniques for situational method engineering, *Information Systems* 24 (3) (1999) 209–228.
- [34] B. Henderson-Sellers, Method engineering for OO system development, *Comm. ACM* 46 (10) (2003) 73–78.
- [35] A.H.M. Ter Hofstede, T.F. Verhoef, On the feasibility of situational method engineering, *Information Systems* 22 (1997) 401–422.
- [36] J. Ralyté, Reusing scenario based approaches in requirement engineering methods: CREWS method base, *Proceedings of the 10th International Workshop on Database and Expert Systems Applications (DEXA'99)*, 1st International Workshop on the Requirements Engineering Process – Innovative Techniques, Models, Tools to support the RE Process (REP'99), Florence, Italy, 1–3 September 1999, IEEE Computer Society, Los Alamitos, CA, USA, 1999, pp. 305–309.
- [37] M. Saeki, CAME: the first step to automated software engineering, *Process Engineering for Object-Oriented and Component-Based Development*. *Procs. OOPSLA 2003 Workshop*, Centre for Object Technology Applications and Research, Sydney, Australia, 2003, pp. 7–18.

- [38] J.-P. Tolvanen, M. Rossi, H. Liu, Method engineering: current research directions and implications for future research, in: *Method Engineering, Principles of Method Construction and Support*, Chapman-Hall, London, 1996, pp. 296–317.
- [39] V.P. Nguyen, B. Henderson-Sellers, Towards automated support for method engineering with the OPEN approach, in: *Procs. 7th IASTED International Conference on Software Engineering and Applications*, ACTA Press, Anaheim, CA, USA, 2003, pp. 691–696.
- [40] J.-P. Tolvanen, *Incremental Method Engineering with Modeling Tools: Theoretical Principles and Empirical Evidence*, Ph.D. Thesis, University of Jyväskylä, Finland, 1998.
- [41] J. Ralyté, C. Rolland, An assembly process model for method engineering, in: *Procs. CAiSE 2001LNCS 2068*, Springer-Verlag, Berlin, 2001, pp. 267–283.
- [42] M. Bajec, M. Krisper, R. Rupnik, The scenario for constructing flexible, people-focused systems development methodologies, *Procs. ECIS 2004* (2004).